CHAPTER 15: THE SAFETY STACK, AND WHERE THE GUARDRAILS SIT

Summary: *A guardrail that depends on the model choosing to respect it is not a guardrail, it is a preference with good intentions. This chapter sets out the two bars a real control has to clear, why defence in depth is a selection criterion rather than a slogan, what separates a failure mode intrinsic to these systems from an ordinary bug that happens nearby, and the documented mechanics of permission modes, isolation and refusals that scenarios are built from.*

Two Bars a Guardrail Must Clear

When a scenario asks which measures constitute real guardrails, two bars separate the answers from the decorations.

Somebody can **show** that a **failure** of this kind gets **caught before anything ships**. Something ran, something scored it, a human would have known.

There is a **point of control**, sitting **outside the model**, that is able to **turn the action down**. A place in the system rather than in the prompt, where the thing simply does not happen.

Three things fail those bars and appear constantly. **After-the-fact record keeping** documents what **already occurred**. **Prompt hygiene** improves the **odds** and **enforces nothing**. **Self-reported anecdote**, meaning somebody tried it and it seemed fine, is not **evidence of coverage**.

Name the condition this diagnoses: the **unencoded policy** is a rule that exists in intent, in prose, in a conversation or in a system prompt, but **nowhere in machinery**. It holds exactly as long as the model **chooses to comply** with it, which means it holds right up until the moment it matters.

Defence in Depth Is a Selection Criterion

When a scenario asks you to pick from a list of protective tactics, the property being selected for is specific and worth stating as a test.

Does this **keep working** after another layer has already been **defeated**?

That question sorts the list fast. A control that only functions while everything else is intact is a **single point of failure** wearing protective clothing. And a knob that alters the *style* of output, tone, verbosity, hedging, formality, is **not enforcing policy** at all, however much it changes what the answers look like.

The same idea gives you the two-part answer to injection scenarios. Only two kinds of measure act on the attack itself:

Make it **less likely** that **fetched text** gets **followed** as though it were an **order**. Delivery channel, framing, vetting.

Bound what a **successful injection** can **reach**. Least privilege, narrow scopes, sandboxing.

Notice that the second assumes the first fails. That is the point. Everything else on those lists either **misattributes immunity** to model size or sampling settings, which confer none, or operates only **after the damage is done**.

Keep Privilege Decisions Out of the Channel the Attacker Controls

This is the single most transferable idea in the chapter.

When content arrives from outside, a fetched page, an uploaded document, an email, an attacker may control that text. Any **decision about privilege** that is made inside that channel is a decision the **attacker can influence**.

So the correct remedy always **alters** what **authority** fetched text is allowed to **carry** once it is inside the prompt. It does not trim the text, vet it, wrap it in encryption, or reformat it while leaving it holding exactly the authority it held before. Those all operate around a boundary that was never actually enforced.

The documented defences are structural in exactly this sense.[1] Deliver untrusted content where it will be treated with scepticism. Screen tool output by passing the raw output to a small classifier call, using structured outputs so the verdict is a parseable value your application can branch on. Apply least privilege so a successful injection does minimal damage. Withhold secrets the model does not need. Run tools sandboxed with narrowly scoped permissions. Red-team the workflow before deployment with documents, emails and tool outputs that deliberately contain

injection attempts. And analyse outputs regularly for signs of successful injection, to refine prompts, validation and filtering.

Against a deliberately adversarial *user*, rather than adversarial *content*, the documented set differs and is worth keeping separate: pre-screen input with a lightweight model constrained by structured outputs to a simple classification before it reaches the main conversation; filter for known injection patterns, including using a model given known jailbreaking language as examples to build a generalised validation screen; write system prompts that emphasise ethical and legal boundaries and explicitly say how to refuse; and **respond to repeat offenders** by adjusting responses, **throttling or banning** them.[1]

That last one is the one people forget. Some of the response to abuse is **account-level**, not **prompt-level**.

A “Never” Guarantee Has Only Two Places to Live

When a requirement is absolute, something must never be sent, never be disclosed, never be committed, count the controls that actually sit on that path. There are two.

A **control** on what can be **drafted**. Constraining what the system is able to produce at all.

A **control** on what gets **authorised to send**. A gate the output must pass through before it leaves.

Everything else in the usual list improves the quality of drafts, or detects failures after delivery. Both are worth having and neither can support the word “**never**”, because one makes a bad outcome less likely and the other tells you it happened.

The same arithmetic applies to a preventive egress boundary. What decides where **data may travel** is a **boundary** that **refuses the traffic**, not a policy that describes which traffic would be acceptable.

Safety at a Scope Boundary

A recurring scenario: the system must behave differently, or stop, when a request crosses out of its intended scope. Somebody describes an emergency to a triage assistant. Somebody asks a support bot for legal advice.

Two properties make that safe, and both are required.

Something other than the model **spots the condition** and **sends it somewhere**, with no dependence on the model **deciding to cooperate**. An instruction to escalate is a request. A detector plus a route is a mechanism.

The **route** is **demonstrated by evaluation** rather than **assumed**. Somebody constructed the boundary cases and confirmed they land where they should.

The **named anti-pattern** is the escalation that only happens if the model **decides to escalate**. It reads as responsible design and it is the unencoded policy again, in a costume that fools reviewers.

What Counts as an Intrinsic Failure Mode

When a scenario asks for risks and failure modes of these systems **specifically**, the discriminating question is cleaner than it looks.

Strip the model out of this system entirely. Does the **same thing still happen**?

If yes, it is an ordinary defect that happens to occur near a model: a timeout, a permissions misconfiguration, a schema mismatch, a bad deployment. Real, worth fixing, and not what the objective is about.

If no, it is intrinsic. The **intrinsic** ones are things the **model emitted on its own**, with every surrounding component doing its **job properly**. A fabricated citation with the retrieval working fine. A confidently wrong classification with the tools returning accurate data. A refusal on legitimate work. Those cannot happen in a system without a model, and they are the ones your existing engineering practice has no answer for.

Exfiltration Needs Both Halves

The word gets used loosely and the exam uses it precisely. **Exfiltration** requires **two things at once**:

Text an **attacker wrote** gets into the **channel the model reads instructions** from.

There is a **way out**, able to **move data** somewhere **nobody approved**.

One without the other is something else. Hostile text with no outbound path is an attempted manipulation. An outbound path with no hostile instruction is normal authorised use. The **pairing** is what makes it **exfiltration**, and a scenario offering only one half is describing a different risk.

Risk scoring has a similar structure. When a scenario asks what must happen before risks can be scored, the inputs are what is at **stake** and who could **reach** it. Everything else in those lists operates on ratings that do not exist yet.

Permission Modes, and the Boundary That Can Be Forgotten

Permission modes govern which actions run without asking, and the documented mechanics produce several items.[2]

What the **reviewing classifier sees**: user messages, non-read-only tool calls, and project instruction file content. **Tool results are stripped**, precisely so hostile content in a file or web page cannot manipulate the classifier directly. Separately, a **server-side probe** scans incoming tool results and **flags suspicious content** before Claude reads it.

Now the part worth memorising, because it is a genuine trap:

A **boundary stated in conversation** is treated as a **block signal** and stays in force until lifted, but it is **not stored as a rule**. The classifier re-reads it from the transcript on every check. So the boundary can be **lost** if **context compaction** removes the message that stated it. The documented recommendation for a hard guarantee is to add a deny rule.

That is the unencoded policy with a precise mechanism attached. A spoken constraint lives in the transcript; a deny rule lives in configuration. Only one of them survives compaction.

The rest of the mechanics:

Behaviour	Detail
Deny rules	**Block in every mode**, including bypassPermissions
Allow rules	Have **no effect** in bypassPermissions
Some actions	**Auto-approved by no mode** at all, including removals targeting critical paths
Classifier produces no verdict	The action is **denied**. This happens when a separate safety check refused its own request, or the response did not parse
Repeated blocks	**3 in a row or 20 in total** pauses the mode and Claude Code resumes prompting. The thresholds are **not configurable**
Non-interactive run with no prompt available	The action does **not run** and the **work continues**
Auto mode selected but unavailable	The session starts in **Manual**

Two of those deserve emphasis. **Failure of the classifier denies** rather than permits, which is fail-closed and the right default. And **deny beats everything** while **allow does not**, so a security posture built on allow rules has a mode in which it evaporates.

Isolation Is a Different Control From Permission

These get conflated and the documentation separates them cleanly.[3]

Permission modes decide whether a **tool call runs** at all, and whether the **user is asked first**.

Isolation restricts what a command can **access** once it is **already running**.

The **action-reviewing classifier** is a **per-action control**, not an **isolation boundary**. It inspects individual actions; it does not contain them.

And one specific, testable gap: the **built-in sandboxed Bash tool** restricts **only Bash commands**. Built-in file tools, MCP servers and hooks continue to run directly on the host. Every other approach documented on that page places the **whole** Claude Code process inside the isolation boundary. A sandbox that contains **one tool** is **not a sandbox around the system**.

Refusals: Categories, Nulls, and the Reset

Refusals carry structure worth knowing exactly.[4]

A classifier refusal returns a stop details object naming the **policy category** that triggered it, plus a human-readable explanation. That explanation's text is **not stable** and should be **displayed rather than parsed**.

The **category field is null** where a decline belongs to **no named policy area**, and that null is a **normal, permanent value** rather than a placeholder. Code that treats null as missing data, or retries expecting a category to appear, is coding against a misunderstanding.

Two documented categories carry a warning attached. **Cyber** covers requests that could enable cyber harm such as malware or exploit development, and **benign cybersecurity work** can also trigger it. **Bio** covers requests that could enable biological harm, and **beneficial life sciences work** can also trigger it. So a refusal in these areas is not evidence that the user was doing something wrong.

A refusal can arrive **before any output**, or **mid-stream after partial output**. Partial output is to be treated as **incomplete and discarded**.

And the operational rule that catches people: on a **refusal stop reason**, the **conversation context must be reset** before continuing, either by removing or rephrasing the turn that triggered it, or by clearing the history. **Continuing without resetting** produces **continued refusals**.[5] A retry loop with no reset is an infinite loop with a bill attached.

Scanning Skills, and What Never Gets Scanned

For Claude Enterprise organisations, automated security scanning for custom Skills is available in beta, checking for signs of malicious behaviour such as hidden code execution, sending data to an outside service, or instructions that tamper with Claude's safeguards.[6]

The outcomes are graded: a Skill that **fails the scan**, or whose scan has **not finished**, is **blocked from use**. One that **passes with a warning** stays **usable behind a caution notice**. And the scope is limited by design: **scanning complements** the review checklist and does **not replace** it.

The exclusions are where the items live. **Skills uploaded through the API** are **not scanned**. Skills **already in the organisation** when scanning was turned on are **not scanned**. And organisations with certain **data handling configurations** are **not scanned**, including **customer-managed encryption keys**, **zero data retention**, and **HIPAA readiness**.

Read that last one carefully, because it is counter-intuitive and therefore examinable. The organisations with the **strongest data handling requirements** are the ones that do **not get automated scanning**. The privacy guarantee and the scanning service are in tension, and the resolution is that the guarantee wins, which means those organisations carry more of the review burden manually.

One governance rule from the same page is worth carrying everywhere: **Skill authors** should **not be their own reviewers**.

What a Skill Can Enforce, and What Has to Sit in the Application

Scanning asks whether a Skill is safe to run. A different question decides where a guardrail belongs before anybody scans anything, and it turns on what a Skill is **able to guarantee**.

A Skill **shapes what the model produces**. It carries instructions the model reads, so it can require a disclaimer, forbid a category of claim, fix a house format, insist that an assertion arrives with a citation attached. **Content constraints belong there**, and

putting them there is cheap, revisable and visible to the people who own the content.

A Skill **cannot authenticate a caller**, and it **cannot guarantee that a record was written**. It is **instruction rather than mechanism**, and instruction is **followed rather than enforced**. So **authorisation stays in the application**, along with anything that has to be **durably recorded**: the audit entry, the committed transaction, the log somebody will read in a dispute six months from now.

The rule compresses to one line. If the requirement is about **what the answer says**, a Skill can hold it. If the requirement is about **who was allowed to ask**, or **what must survive the request**, it needs **code that runs whether or not the model cooperates**.

The **named anti-pattern is the instructed guarantee**: an authorisation rule or an audit obligation written into a Skill, where it reads exactly like a control, satisfies a reviewer skimming for one, and enforces nothing.

Oversight That Scales With Consequence

A recurring scenario asks how to review a growing volume of changes without either rubber-stamping everything or blocking the organisation.

The workable model has two halves. **Published, reusable standards** that everyone adopts, so most changes are governed without a review event. And review that fires only when a **written classification of risk** says it should, so scrutiny goes where the consequence is.

That combination makes oversight scale with **consequence** rather than with **volume**, which is the property being selected for. A model that reviews everything scales with volume and collapses. A model that reviews nothing scales beautifully and governs nothing.

How Wrong Answers Are Built on Safety Questions

- A **policy with no machinery behind** it. The option states a rule in a prompt, a document or a conversation. The test: what **physically stops** this if the model does not comply?

- A **control that dies with the layer above** it. The test: does this **still work** after something else has **already failed**?
- **Detection offered where prevention was required**. The test: does the scenario say "**never**", or does it ask how you would **find out**?
- A **style knob offered as policy enforcement**. The test: does this change what the **system** may do, or only how it **sounds**?
- An **ordinary defect offered as an intrinsic failure mode**. The test: **strip the model out** and ask whether this still happens.
- **Half of an exfiltration**. The test: is there both **hostile text** reaching the instruction channel and an **unauthorised outbound path**?

When the scenario says	The answer is usually	The trap is usually
Which measures are real guardrails, two wanted	Proof the failure is caught before shipping, and a control point outside the model able to turn it down	Audit logs, prompt wording, or somebody having tried it
Protective tactics must be chosen	The ones that survive another layer being defeated	Single points of failure, or knobs that change output style
Retrieved content is being obeyed as instruction	Alter what authority fetched text carries in the prompt	Shrinking, screening or encrypting around an unenforced boundary
Injection must be addressed, two wanted	Make fetched text less likely to be followed as an order, and bound what a success reaches	Larger models, lower temperature, or post-hoc detection

When the scenario says	The answer is usually	The trap is usually
Something must never leave the system	A constraint on what can be drafted, and a gate on what is authorised to send	Better drafts, or detection after delivery
A request crosses out of scope	Something other than the model spots it and routes it, proven by evaluation	An instruction telling the model to escalate
A failure mode of these systems is asked for	Something the model itself emitted while every other part behaved	A timeout, a misconfiguration, or a schema mismatch
Exfiltration is alleged	Attacker-written text reaching the instruction channel, and a route out to somewhere unapproved	One half without the other
Risks must be scored	What is at stake, and who could reach it	Anything operating on ratings that do not exist yet
A boundary was agreed in conversation	Add a deny rule; a spoken boundary is re-read from the transcript and can be lost to compaction	Trusting the stated boundary to persist
bypassPermissions is in use	Deny rules still block; allow rules do nothing	Assuming allow rules still grant, or that deny is bypassed

When the scenario says	The answer is usually	The trap is usually
The reviewing classifier returns no verdict	The action is denied	Assuming a missing verdict permits
The classifier keeps blocking	3 consecutive or 20 total pauses the mode; not configurable	Expecting to tune the thresholds
A sandboxed Bash tool is in place	Only Bash is restricted; file tools, MCP servers and hooks run on the host	Treating it as isolation for the whole process
A refusal category is null	Normal and permanent, not missing data	Retrying until a category appears
A refusal mentions cyber or bio	Benign security and life sciences work can trigger these	Concluding the user was doing something illegitimate
A refusal arrives mid-stream	Discard the partial output and reset the context before continuing	Retrying on the same context, which refuses again
Custom Skills need security review	Scanning complements the checklist; authors do not review their own Skills	Assuming every Skill is scanned
The organisation uses customer-managed keys, zero retention or HIPAA readiness	Automated Skill scanning does not apply; review manually	Assuming stronger data controls mean stronger scanning

When the scenario says	The answer is usually	The trap is usually
Review must scale across many changes	Published reusable standards, plus review fired by a written risk classification	Reviewing everything, or reviewing by sampling volume
Failure modes for a risk review, the inherent ones wanted, two wanted	Fluent output that is wrong, and two runs of one request answering differently	Budget growth, network dependence, or needing a valid credential
Customer documents are ingested by an assistant holding outbound tools	Contain the content structurally and gate anything that sends or writes	A larger model, an instruction in the system prompt alone, or matching known attack phrasing

A guardrail has to be placed in a Skill or in the application | Content constraints in the Skill; authorisation and durable records in the application | An authorisation rule or audit obligation written into a Skill, which instructs rather than enforces |

The habit worth carrying: for every safety claim in a design, **point at the component that enforces** it. If your finger lands on a paragraph of English, it is not enforced.

What to Remember

- A guardrail clears **two bars**: somebody can show this **failure gets caught before anything ships**, and there is a **point of control outside the model** able to **turn the action down**. Audit records, prompt hygiene and “we tried it and it was fine” **clear neither**.
- The **unencoded policy** is a rule that exists in intent or prose but **nowhere in machinery**. It holds only while the model **chooses to comply**.
- **Defence in depth** is a **selection test**: does this control **still work** after another layer has been **defeated**? A control that needs everything else

intact is a single point of failure, and a knob that changes **output style** enforces **no policy**.

- Against injection, only **two measures act on the attack**: making it **less likely** that fetched text gets **followed as an order**, and **bounding what a success can reach**. Model size and sampling settings confer **no immunity**.
- Any privilege decision made **inside a channel the attacker controls** is a decision the **attacker influences**. The remedy **alters what authority** fetched text is allowed to **carry**, rather than trimming it, vetting it or wrapping it in encryption while the **boundary itself still enforces nothing**.
- A “never” requirement has **exactly two places to live**: a constraint on what can be **drafted**, and a gate on what is **authorised to send**. Better drafts and after-the-fact detection **cannot support the word**.
- Safety at a scope boundary needs **something other than the model** to **spot the condition** and **send it somewhere**, with no dependence on the model **cooperating**, and with that route **proven by evaluation**.
- An **intrinsic failure mode** is something the **model emitted on its own**, with every surrounding component doing its **job**. The test: **strip the model out** and ask whether it still happens. If it does, this is an **ordinary defect standing nearby**.
- Exfiltration needs **both halves**: **text an attacker wrote** reaching the channel the model takes instructions from, and a **way out** able to move data somewhere nobody approved. **One half alone** is a different risk. Risk scoring needs what is at **stake** and who could **reach** it before any rating exists.
- The action-reviewing classifier sees **user messages, non-read-only tool calls and project instruction files**; **tool results are stripped**, and a **server-side probe** flags suspicious tool results **before Claude reads** them.

- A boundary stated in conversation is **re-read from the transcript** on every check and is **not stored as a rule**, so **compaction can lose** it. **Add a deny rule** for a hard guarantee.

- **Deny rules block in every mode** including bypassPermissions; **allow rules have no effect** there; some actions are **auto-approved by no mode at all**. When the classifier produces **no verdict** the action is **denied**, and **3 consecutive or 20 total blocks** pauses the mode, with thresholds that **cannot be configured**. In a non-interactive run the action does **not run** and the **work continues**, and **auto mode that is unavailable** starts the session in **Manual**.

- **Permission modes** decide whether a call **runs**; **isolation** restricts what it can **reach** once running; the classifier is a **per-action control** and **not an isolation boundary**. The built-in sandboxed Bash tool **restricts only Bash**, leaving **file tools, MCP servers and hooks on the host**.

- A refusal returns a **policy category** plus an explanation whose text is **unstable** and should be **displayed rather than parsed**. A **null category** is **normal and permanent**. **Cyber and bio** categories can trigger on **benign security and life sciences work**. **Partial output** before a mid-stream refusal is **discarded**, and the **context must be reset** before continuing or refusals will repeat.

- Skill scanning **blocks** a Skill that **fails or has not finished** scanning and **allows one that passes with a warning** behind a caution notice, and it **complements rather than replaces** the review checklist. **Skills uploaded via the API**, **Skills predating the feature**, and organisations using **customer-managed keys, zero data retention or HIPAA readiness** are **not scanned**. Authors do **not review their own Skills**.

- **Oversight scales with consequence, not volume**: **published reusable standards** everyone adopts, plus **review that fires** only when a **written classification of risk** says it should.

- A **Skill shapes what the model produces**, so **content constraints live there**: disclaimers, forbidden claims, required citations, house format. A Skill **cannot authenticate a caller or guarantee a record was written**, so **authorisation and durable recording stay in the application**, where **code runs whether or not the model cooperates**.